CultivaX — Construction Phase Workflow

March 1–30, 2026 | Day-by-Day Development Plan

Team Members & Work Distribution

Member	GitHub ID	Role Focus	Share
Arpit	`malikarpit40@gmail.com`	Core backend, DB, Event System, Replay Engine, Deployment, some Frontend	**40%**
Ayush Kumar Meena	`Ayushmeena7027@gmail.com`	Authentication, User Management, Admin APIs, Testing	**15%**
Ravi Patel	`ravipatel7570849190@gmail.com`	SOE Module (Providers, Trust Score, Marketplace)	**15%**
Prince	`princenagar2904@gmail.com`	Frontend (Next.js pages, components, UI/UX)	**15%**
Shivam Yadav	`Shivam1535ly@gmail.com`	ML Module, Media Handling, Workers, Testing	**15%**

Tech Stack (Google Cloud)

- **Backend:** Python 3.11 + FastAPI
- **Database:** PostgreSQL (Cloud SQL)
- **Frontend:** Next.js 14 (React PWA)
- **Deployment:** Google Cloud Run
- **Storage:** Google Cloud Storage
- **Auth:** JWT (internal)
- **ORM:** SQLAlchemy + Alembic migrations

Git Conventions

- **Branching:** `main` ← `develop` ← `feature/*`
- **Commit format:** `type: description` (feat/fix/docs/chore/test/deploy)
- **Each commit = one logical unit of work**

WEEK 1 — Foundation & Database (Mar 1–7)

Day 1 — Saturday, March 1

Arpit

Task: Initialize Git repo, README, project root structure

```
Files to create:
  README.md
  .gitignore
  docs/           (empty, will hold documentation)
```

Commit 1: `chore: initialize git repository` **Commit 2:** `docs: add project README with overview and tech stack`

README.md must include:

- Project name & description (CultivaX — AI-powered crop lifecycle management)
- Tech stack list
- Team members
- Setup instructions placeholder
- License

Prince

Task: Initialize Next.js frontend project

```
cd /path/to/cultivax
npx -y create-next-app@latest frontend --ts --tailwind --app --src-dir --
no-import-alias
```

```
Files created:
  frontend/package.json
  frontend/tsconfig.json
  frontend/src/app/layout.tsx
  frontend/src/app/page.tsx
  frontend/next.config.js
```

Commit: `chore: initialize nextjs frontend project`

Day 2 — Sunday, March 2

Arpit

Task: Initialize FastAPI backend project structure

```
Files to create:
  backend/app/__init__.py
  backend/app/main.py              ← FastAPI app entry point
  backend/app/config.py            ← Settings via pydantic-settings
  backend/app/database.py          ← SQLAlchemy engine + session
  backend/requirements.txt         ← fastapi, uvicorn, sqlalchemy,
alembic, psycopg2-binary, pydantic, python-jose, passlib, python-
multipart, httpx, google-cloud-storage
  backend/Dockerfile
  backend/.env.example
```

main.py skeleton:

```python
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI(title="CultivaX API", version="0.1.0")
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=
["*"], allow_headers=["*"])

@app.get("/health")
async def health():
    return {"status": "ok"}
```

Commit: `feat: initialize fastapi backend project structure`

Ayush

Task: Add global error handling middleware, CORS config, and idempotency guard

```
Files to create:
  backend/app/middleware/__init__.py
  backend/app/middleware/error_handler.py    ← global exception handler,
structured error responses
  backend/app/middleware/idempotency.py      ← Idempotency-Key header
check, reject duplicates (MSDD 8.13)
```

Commit: `feat: add global error handling and idempotency middleware`

Ayush

Task: Create security utilities module

```
Files to create:
  backend/app/security/__init__.py
```

```
  backend/app/security/auth.py     ← password hashing (passlib), JWT
create/verify (python-jose)
```

Commit: `feat: add jwt and password hashing utilities`

Day 3 — Monday, March 3

Arpit

Task: Set up Alembic migrations + create users table

```
cd backend && alembic init alembic
```

```
Files to create/modify:
  backend/alembic/env.py           ← configure with SQLAlchemy metadata
  backend/alembic/script.py.mako   ← Alembic migration template
  backend/alembic.ini              ← DB URL from env
  backend/app/models/__init__.py
  backend/app/models/user.py       ← User model (user_id UUID PK,
full_name, phone, role, password_hash, region, preferred_language,
accessibility_settings JSONB, is_active, is_onboarded, created_at,
deleted_at, deleted_by)
  backend/alembic/versions/001_create_users_table.py
```

Soft Delete Policy (MSDD 5.10): All models must include `is_deleted`, `deleted_at`, `deleted_by` — **no hard deletes** on any major table. **Commit 1:** `chore: configure alembic for database migrations` **Commit 2:** `feat: add user model and initial migration`

Shivam

Task: Create base model utilities and common schemas

```
Files to create:
  backend/app/models/base.py       ← Base declarative model with UUID PK
mixin, timestamps
  backend/app/schemas/__init__.py
  backend/app/schemas/common.py    ← ResponseModel, PaginationParams,
ErrorResponse
```

Commit: `feat: add base model mixins and common schemas`

Day 4 — Tuesday, March 4

Arpit

Task: Create CTIS core database tables

```
Files to create:
  backend/app/models/crop_instance.py   ← crop_instances table (all fields
from TDD Section 2.3.1)
  backend/app/models/action_log.py      ← action_logs table (TDD Section
2.3.2)
  backend/app/models/snapshot.py        ← crop_instance_snapshots table
(TDD 2.3.3)
  backend/app/models/deviation.py       ← deviation_profiles table (TDD
2.3.4)
  backend/app/models/yield_record.py    ← yield_records table (TDD 2.3.5)
  backend/alembic/versions/002_create_ctis_tables.py
```

Commit: `feat: add CTIS core database models and migration`

Ravi

Task: Create SOE database tables (including equipment, labor, request events)

```
Files to create:
  backend/app/models/service_provider.py  ← service_providers table (TDD
2.5.1) + crop_specializations JSONB
  backend/app/models/equipment.py         ← equipment table (TDD 5.5.2:
equipment_id, provider_id FK, type, availability, hourly_rate, is_flagged)
  backend/app/models/labor.py             ← labor table (MSDD 2.6:
labor_id, provider_id FK, labor_type, available_units, daily_rate, region,
is_flagged, created_at, updated_at, is_deleted)
  backend/app/models/service_request.py   ← service_requests table (TDD
2.5.2) + provider_acknowledged_at field
  backend/app/models/service_review.py    ← service_reviews table (TDD
2.5.3) — immutable, no hard delete
  backend/app/models/provider_availability.py ← provider_availability
table (TDD 5.3)
  backend/app/models/service_request_event.py ← service_request_events
table (SOE Enhancement 7: event_type, previous_state, new_state,
timestamp, actor_role, request_id FK)
  backend/alembic/versions/003_create_soe_tables.py
```

Commit: `feat: add SOE database models with equipment, labor, and migration`

Day 5 — Wednesday, March 5

Arpit

Task: Create Event System + Admin + Feature Flags tables

```
Files to create:
  backend/app/models/event_log.py       ← event_log table (TDD 2.4.1) with
event_hash UNIQUE
  backend/app/models/admin_audit.py     ← admin_audit_log table (TDD
2.8.1)
  backend/app/models/feature_flag.py    ← feature_flags table (ml_enabled,
clustering_enabled, soe_enabled)
  backend/app/models/abuse_flag.py      ← abuse_flags table (MSDD Patch
4.1)
  backend/alembic/versions/004_create_event_admin_flags_tables.py
```

Commit: `feat: add event log, admin audit, feature flags, and abuse detection models`

Shivam

Task: Create ML + Media + Analytics tables

```
Files to create:
  backend/app/models/ml_model.py        ← ml_models table (TDD 2.6.1) +
evaluation_metrics JSONB, training_dataset_reference
  backend/app/models/ml_training.py     ← ml_training_audit table (TDD
2.6.2) + dataset_source, date_range, feature_schema, preprocessing_version
(ML Enhancement 8: Feature Lineage Tracking)
  backend/app/models/media_file.py      ← media_files table (TDD 2.7.1) +
scheduled_deletion_at, analysis_status
(Pending/Processing/Analyzed/Failed)
  backend/app/models/stress_history.py  ← stress_history table (TDD 5.6.1:
stress_id, crop_instance_id, stress_score, stage, computed_at)
  backend/app/models/regional_cluster.py ← regional_clusters table (TDD
5.6.2: cluster_id, crop_type, region, avg_delay, avg_yield, sample_size,
std_dev_yield, confidence_interval)
  backend/app/models/pest_alert_history.py ← pest_alert_history table
(MSDD 6 Enhancement: pest_id, crop_instance_id, pest_type, alert_level,
detected_by, confidence, created_at)
  backend/alembic/versions/005_create_ml_media_analytics_tables.py
```

Commit: `feat: add ML, media, and analytics database models`

Arpit

Task: Add regional sowing calendars table

```
Files to create:
  backend/app/models/sowing_calendar.py  ← regional_sowing_calendars table
(Patch Module 5: crop_type, region, optimal_start DATE, optimal_end DATE,
version_id)
  backend/alembic/versions/006_create_sowing_calendar_table.py
```

Commit: `feat: add regional sowing calendar model for seasonal window assignment`

Prince

Task: Frontend auth context and protected routes

```
Files to create:
  frontend/src/context/AuthContext.tsx   ← JWT storage, login/logout, user
state
  frontend/src/components/ProtectedRoute.tsx ← redirect to login if not
authenticated
  frontend/src/lib/auth.ts              ← token helpers (get, set, remove
from localStorage)
```

Commit: `feat: add authentication context and protected route wrapper`

Day 6 — Thursday, March 6

Arpit

Task: Create Pydantic schemas for CTIS models

```
Files to create:
  backend/app/schemas/crop_instance.py  ← CropInstanceCreate,
CropInstanceResponse, CropInstanceUpdate, SowingDateUpdate, CropListFilter
  backend/app/schemas/action_log.py     ← ActionLogCreate,
ActionLogResponse
  backend/app/schemas/yield_record.py   ← YieldSubmission, YieldResponse
```

Commit: `feat: add pydantic schemas for CTIS entities`

Ayush

Task: Create Pydantic schemas for user and auth

```
Files to create:
  backend/app/schemas/user.py      ← UserCreate, UserLogin, UserResponse,
TokenResponse
  backend/app/schemas/admin.py     ← AdminAuditResponse
```

Commit: `feat: add pydantic schemas for auth and user entities`

Ravi

Task: Create Pydantic schemas for SOE

```
Files to create:
  backend/app/schemas/service_provider.py  ← ProviderCreate,
ProviderResponse, ProviderFilter
  backend/app/schemas/equipment.py         ← EquipmentCreate,
EquipmentResponse
  backend/app/schemas/service_request.py   ← ServiceRequestCreate,
ServiceRequestResponse
  backend/app/schemas/service_review.py    ← ReviewCreate, ReviewResponse
  backend/app/schemas/labor.py             ← LaborCreate, LaborResponse
```

Commit: `feat: add pydantic schemas for SOE entities`

Day 7 — Friday, March 7

Arpit

Task: Docker & docker-compose setup for local dev

```
Files to create:
  docker-compose.yml     ← postgres:15 + backend + frontend services
  backend/Dockerfile     ← Python 3.11, pip install, uvicorn
  backend/.dockerignore   ← exclude venv, __pycache__, .git, .env
  frontend/Dockerfile    ← Node 18, npm install, next dev
```

Commit: `chore: add docker-compose for local development`

Prince

Task: Frontend global layout, theme, and navigation shell

```
Files to create/modify:
  frontend/src/app/layout.tsx        ← root layout with sidebar nav
  frontend/src/app/globals.css       ← CSS variables, dark theme,
typography
  frontend/src/components/Sidebar.tsx
  frontend/src/components/Header.tsx
```

Commit: `feat: add frontend layout with sidebar navigation`

WEEK 2 — Core Backend APIs (Mar 8–14)

Day 8 — Saturday, March 8

Ayush

Task: User registration + login API endpoints

```
Files to create:
  backend/app/api/__init__.py
  backend/app/api/v1/__init__.py
  backend/app/api/v1/auth.py       ← POST /register, POST /login (returns
JWT)
  backend/app/api/deps.py          ← get_db, get_current_user dependency
```

Commit: `feat: implement user registration and login endpoints`

Prince

Task: Frontend login & registration pages

```
Files to create:
  frontend/src/app/login/page.tsx
  frontend/src/app/register/page.tsx
  frontend/src/lib/api.ts           ← axios/fetch wrapper with base URL,
auto-attach JWT header
  frontend/.env.example             ← NEXT_PUBLIC_API_URL and other
required env vars
```

Commit: `feat: add login and registration pages`

Shivam

Task: Add crop rule template seed data files (wheat, rice, cotton)

```
Files to create:
  backend/data/crop_rules/wheat.json   ← stage definitions, durations,
drift limits, risk params
  backend/data/crop_rules/rice.json
  backend/data/crop_rules/cotton.json
```

Commit: `chore: add crop rule template seed data for wheat, rice, cotton`

Day 9 — Sunday, March 9

Arpit

Task: Crop Instance CRUD endpoints + seasonal window assignment

```
Files to create:
  backend/app/api/v1/crops.py      ← POST /crops (create), GET /crops
(list, paginated, filterable by state/crop_type/include_archived), GET
/crops/{id}, PUT /crops/{id}, PUT /crops/{id}/sowing-date
  backend/app/services/__init__.py
  backend/app/services/ctis/__init__.py
  backend/app/services/ctis/crop_service.py  ← business logic for crop
CRUD
  backend/app/services/ctis/seasonal_window.py ← assign Early/Optimal/Late
by comparing sowing_date to regional_sowing_calendars (MSDD 1.9 + Patch
Module 5)
```

Seasonal Window Assignment (Patch Module 5):

```
def assign_seasonal_window(sowing_date, crop_type, region):
    calendar = get_sowing_calendar(crop_type, region)  # fallback to
national default if missing
    if sowing_date < calendar.optimal_start: return 'Early'
    elif sowing_date <= calendar.optimal_end: return 'Optimal'
    else: return 'Late'
    # Persist at creation — never changes (even if sowing_date modified)
```

Commit: `feat: implement crop instance CRUD with seasonal window assignment`

Ayush

Task: JWT middleware + role-based access guard

```
Files to create/modify:
  backend/app/api/deps.py          ← add require_role() dependency
  backend/app/security/auth.py     ← add role verification
```

Commit: `feat: add role-based access control middleware`

Day 10 — Monday, March 10

Arpit

Task: Action Logging endpoint + chronological validation

```
Files to create:
  backend/app/api/v1/actions.py    ← POST /crops/{id}/actions (log action)
  backend/app/services/ctis/action_service.py  ← validate chronological
invariants, insert action_log
```

Commit: `feat: implement action logging with chronological validation`

Ravi

Task: Service Provider + Equipment CRUD endpoints

```
Files to create:
  backend/app/api/v1/providers.py  ← POST /providers, GET /providers
(filter by region, crop_type, service_type), GET /providers/{id}
  backend/app/api/v1/equipment.py  ← POST /providers/{id}/equipment, GET
/providers/{id}/equipment
  backend/app/services/soe/__init__.py
  backend/app/services/soe/provider_service.py
```

Commit 1: `feat: implement service provider CRUD endpoints` **Commit 2:** `feat: add equipment management endpoints`

Ayush

Task: Register all API routers in main.py

```
Files to modify:
  backend/app/main.py  ← import and include all API routers (auth, crops,
actions, providers, etc.)
  backend/app/api/v1/router.py  ← centralized router aggregation
```

Commit: `feat: register all api routers and create centralized router`

Day 11 — Tuesday, March 11

Arpit

Task: Event Dispatcher — core implementation

```
Files to create:
  backend/app/services/event_dispatcher/__init__.py
  backend/app/services/event_dispatcher/interface.py    ←
EventDispatcherInterface (abstract)
  backend/app/services/event_dispatcher/db_dispatcher.py ←
DBEventDispatcher — publish(), process_pending_events()
  backend/app/services/event_dispatcher/handlers.py      ← event type →
handler mapping
  backend/app/services/event_dispatcher/event_types.py   ← Formal event
type constants catalog (MSDD 3.3):
    # CTIS Events: ActionLogged, ReplayTriggered, StageChanged,
StressUpdated, YieldSubmitted
    # SOE Events: ServiceRequested, ProviderContacted, RequestCompleted,
```

```
ReviewSubmitted
    # ML Events: RiskComputed, MediaAnalyzed, ClusterUpdated,
ModelRegistered
    # Admin Events: UserRoleChanged, ProviderVerified, ProviderSuspended,
FeatureToggled
    # Notification Events: AlertGenerated, RecommendationCreated,
AlertAcknowledged
```

Key logic:

- `publish()`: insert event_log row with status='Created', compute event_hash
- `process_pending_events()`: SELECT FOR UPDATE SKIP LOCKED, process FIFO per crop_instance
- Idempotency via UNIQUE(event_hash)

Commit: `feat: implement in-process event dispatcher with DB persistence`

Prince

Task: Frontend dashboard page skeleton

```
Files to create:
  frontend/src/app/dashboard/page.tsx       ← main dashboard
  frontend/src/components/CropCard.tsx      ← card showing crop status
  frontend/src/components/StatsWidget.tsx   ← stat counters
```

Commit: `feat: add dashboard page with crop cards`

Day 12 — Wednesday, March 12

Arpit

Task: Replay Engine v1 — the core algorithm

```
Files to create:
  backend/app/services/ctis/replay_engine.py
```

Must implement (from TDD Section 4.4):

```
async def replay_crop_instance(crop_instance_id, db):
    # 1. Acquire row lock (SELECT FOR UPDATE) ← prevent concurrent replay
    # 2. Load latest snapshot (if exists)
    # 3. Load ordered action_logs
    # 4. For each action: validate invariant → apply → update stress →
compute risk → enforce drift
    # 5. Update crop_instance row
```

```
    # 6. Create snapshot if threshold met
    # 7. Commit transaction
    # On failure:
    #   - Revert to last stable checkpoint/snapshot (MSDD 1.18)
    #   - Lock crop_instance, set state = 'RecoveryRequired'
    #   - Log error, notify admin
    #   - Prevent further action logging until resolved
```

Commit: `feat: implement deterministic replay engine with snapshot support`

Ravi

Task: Trust Score Engine

```
Files to create:
  backend/app/services/soe/trust_engine.py  ←
compute_trust_score(provider_id)
```

Formula (from TDD 5.5 + SOE Enhancements 2, 5):

```
trust = w1*CR + w2*(1-CPR) + w3*norm_rating + w4*VB + w5*ConsistencyScore
- w6*EP
```

Where:

- ConsistencyScore = 1 – variance(completion_time, resolution_score) (SOE Enhancement 5)
- Apply temporal decay: trust *= decay_factor (0.98/month inactivity) (SOE Enhancement 2)
- clamp(trust, 0, 1) **Commit:** `feat: implement trust score computation engine`

Day 13 — Thursday, March 13

Arpit

Task: Wire Replay Engine into Event Dispatcher + State Machine enforcement

```
Files to modify:
  backend/app/services/event_dispatcher/handlers.py  ← ActionLogged →
ReplayTriggered handler
  backend/app/services/ctis/action_service.py        ← after action
insert, publish ActionLogged event
Files to create:
  backend/app/services/ctis/state_machine.py  ← CropStateMachine class
```

State Machine must enforce (MSDD 1.5):

```
VALID_TRANSITIONS = {
    'Created': ['Active'],
    'Active': ['Delayed', 'AtRisk', 'ReadyToHarvest'],
    'Delayed': ['Active', 'AtRisk'],
    'AtRisk': ['Active', 'Delayed'],
    'ReadyToHarvest': ['Harvested'],
    'Harvested': ['Closed'],
    'Closed': ['Archived'],
    'RecoveryRequired': ['Active'],  # MSDD 1.18 — admin resolution only
}
def transition(current_state, new_state):
    if new_state not in VALID_TRANSITIONS.get(current_state, []):
        raise InvalidStateTransition()
    # Every transition MUST emit a StageChanged event (MSDD 1.5)
```

Commit 1: `feat: integrate replay engine with event dispatcher` **Commit 2:** `feat: implement crop instance state machine with transition validation`

Shivam

Task: ML Risk Predictor — stub implementation

```
Files to create:
  backend/app/services/ml/__init__.py
  backend/app/services/ml/risk_predictor.py   ←
predict_risk(crop_instance) → returns risk_probability, confidence,
data_sufficiency_index, model_version (ML Enhancement 2: Confidence
Propagation)
```

For now: Use rule-based logic (no trained model yet). Return risk based on stress_score thresholds. **Confidence Propagation (ML Enhancement 2):** Every ML output must include: prediction_value, confidence_score (0-1), data_sufficiency_index, model_version_used. Low confidence → softer recommendation tone. risk_adjusted = prediction × confidence. **Commit:** `feat: add ML risk predictor with rule-based fallback`

Ravi

Task: Escalation Policy engine

```
Files to create:
  backend/app/services/soe/escalation_policy.py  ←
check_escalation(provider_id)
```

Logic (MSDD Enhancement 3):

```
def check_escalation(provider_id):
    if complaint_ratio > THRESHOLD:  # e.g., 20%
        if complaint_count <= WARNING_LIMIT: return 'Warning'
        elif complaint_count <= SUSPEND_LIMIT: return
'TemporarySuspension'
        else: return 'PermanentSuspension'
```

Commit: `feat: implement complaint escalation policy engine`

Day 14 — Friday, March 14

Arpit

Task: Stress Score Integration + Deviation Profile service

```
Files to create:
  backend/app/services/ctis/stress_engine.py
  backend/app/services/ctis/deviation_tracker.py  ← track consecutive
deviations, trend slope
```

Stress Integration (from TDD 4.7):

```
def integrate_stress(backend_ml, weather_risk, deviation_penalty,
edge_signal, previous_stress, confidence):
    signal = w_backend*backend_ml + w_weather*weather_risk +
w_deviation*deviation_penalty + w_edge*edge_signal
    effective = signal * confidence
    new_stress = alpha * effective + (1-alpha) * previous_stress
    # clamp daily jump, bound [0,1]
    return new_stress
```

Deviation Tracker (from MSDD 1.9.1):

```
def update_deviation_profile(crop_instance_id, current_deviation):
    # increment consecutive_deviation_count
    # compute deviation_trend_slope
    # set recurring_pattern_flag if threshold exceeded
```

Commit 1: `feat: implement multi-signal stress score integration` **Commit 2:** `feat: add deviation profile tracking service`

Shivam

Task: Media upload endpoint + Weather API integration

```
Files to create:
  backend/app/api/v1/media.py          ← POST /crops/{id}/media (upload
image/video)
  backend/app/services/media/__init__.py
  backend/app/services/media/upload_service.py  ← save to local storage
(Cloud Storage later), set scheduled_deletion_at (3-month retention per
MSDD 4.6), set analysis_status='Pending'
  backend/app/services/weather/__init__.py
  backend/app/services/weather/weather_service.py  ← fetch from
OpenWeatherMap API, fallback to historical
```

Weather Service (MSDD 4.8):

```python
async def get_weather_risk(region: str) -> float:
    # call OpenWeatherMap/Visual Crossing API
    # parse temperature, rainfall, wind
    # compute weather_risk_score (0-1)
    # fallback to historical baseline if API fails
```

Commit 1: `feat: add media upload endpoint with file handling` **Commit 2:** `feat: add weather api integration with fallback`

WEEK 3 — Features & Frontend (Mar 15–22)

Day 15 — Saturday, March 15

Arpit

Task: Snapshot Manager + Crop Rule Template

```
Files to create:
  backend/app/services/ctis/snapshot_manager.py  ← create/load snapshots,
threshold logic (every N actions per MSDD 1.8.2)
  backend/app/models/crop_rule_template.py       ← crop rules with
stage_definitions, risk_parameters, irrigation_windows,
fertilizer_windows, harvest_windows, version_id, effective_from_date (MSDD
1.4)
  backend/alembic/versions/007_add_crop_rule_templates.py
```

Commit: `feat: add snapshot manager and crop rule template model`

Ayush

Task: Admin user management + provider verification APIs

```
Files to create:
  backend/app/api/v1/admin.py  ← GET /admin/users, PUT
/admin/users/{id}/role, DELETE /admin/users/{id} (soft delete), PUT
/admin/providers/{id}/verify, PUT /admin/providers/{id}/suspend, GET
/admin/audit
```

Commit 1: `feat: add admin user management endpoints` **Commit 2:** `feat: add admin provider verify, suspend, and audit log endpoints`

Day 16 — Sunday, March 16

Arpit

Task: What-If Simulation Engine

```
Files to create:
  backend/app/services/ctis/whatif_engine.py  ← clone crop state in
memory, run replay, return projected state (NO persist)
  backend/app/api/v1/simulation.py            ← POST /crops/{id}/simulate
```

Deep Copy Enforcement (MSDD 1.14): What-If must clone:

- Entire CropInstance state (stage, stress, risk, day_number)
- Stress history
- Deviation history
- Seasonal category
- Market snapshot (if available)
- Weather snapshot Simulation occurs in **isolated memory context**. **No live mutation allowed.**
  Commit: `feat: implement what-if simulation engine`

Prince

Task: Frontend crop list & crop detail pages

```
Files to create:
  frontend/src/app/crops/page.tsx              ← list all crops
  frontend/src/app/crops/[id]/page.tsx         ← crop detail with timeline
  frontend/src/components/CropTimeline.tsx     ← visual timeline component
  frontend/src/components/ActionLogList.tsx    ← action history list
```

Commit: `feat: add crop list and crop detail pages with timeline view`

Day 17 — Monday, March 17

Arpit

Task: Yield Submission flow + Behavioral Adaptation

```
Files to create:
  backend/app/api/v1/yield.py                    ← POST /crops/{id}/yield
  backend/app/services/ctis/yield_service.py     ← verify yield, compute
ml_yield_value, bio cap, set state=Harvested
  backend/app/services/ctis/behavioral_adapter.py ← farmer pattern
detection, bounded offset
```

Key logic: Farmer Truth vs ML Truth separation (TDD 4.9) **Yield Verification (MSDD 1.12 + 4.3):**

```
def submit_yield(crop_instance_id, reported_yield):
    # 1. Compute YieldVerificationScore (0-1) from stress history,
weather, seasonal risk
    # 2. If reported_yield > biological_limit: ml_yield_value =
biological_limit
    # 3. reported_yield NEVER modified in UI (Farmer Truth)
    # 4. ml_yield_value used for training only (ML Truth)
    # 5. Regional cluster updated prospectively only (Regional Truth)
    # 6. Set state = 'Harvested', trigger replay
```

Behavioral Adaptation (MSDD 4.2 Layer 3 + ML Enhancement 6):

```
def compute_behavioral_offset(farmer_id, crop_type):
    # detect if farmer consistently delays/advances actions
    # if pattern recurring > threshold: apply offset (max ±X days)
    # offset resets at season end
    # NEVER modifies baseline template
    # Must be reversible (ML Enhancement 6)
```

Commit 1: `feat: implement yield submission with verification` **Commit 2:** `feat: add behavioral adaptation pattern detection`

Ravi

Task: Service Request lifecycle endpoints

```
Files to create:
  backend/app/api/v1/service_requests.py  ← POST /service-requests, PUT
/{id}/accept, PUT /{id}/complete
  backend/app/services/soe/request_service.py ← matchmaking, state
```

```
transitions, emit ServiceRequestEvent on each state change (SOE
Enhancement 7)
```

Request verification requirement (SOE Enhancement 8): Review eligibility requires ServiceRequest.status = Completed + minimum interaction time threshold + one review per request + time window limit. **Commit:** `feat: implement service request lifecycle endpoints`

Day 18 — Tuesday, March 18

Arpit

Task: Frontend — Crop Creation Form (minimal frontend work)

```
Files to create:
  frontend/src/app/crops/new/page.tsx          ← create crop form
  frontend/src/components/CropForm.tsx         ← form component
  frontend/src/hooks/useApi.ts                 ← custom hook for API
calls with JWT
```

Commit: `feat: add crop creation form page`

Ravi

Task: Exposure Fairness Algorithm + Fraud Detector

```
Files to create:
  backend/app/services/soe/exposure_engine.py
  backend/app/services/soe/fraud_detector.py   ← detect review pattern
anomaly, rating spike, IP correlation
```

Exposure Formula (from MSDD 2.8.3 + SOE Enhancement 1):

```
ranking = trust_score * 0.85 + random_factor * 0.10 + regional_weight *
0.05
```

Additional constraints (SOE Enhancement 1, 9, 11):

- Top 3 providers cannot occupy >70% exposure over rolling 30 days
- If provider consistently dominates visibility, slight exposure decay applied
- Regional Saturation Control: if too many providers in one micro-region, rank by trust with minimum exposure rotation (SOE Enhancement 11)
- Trust Score Transparency: Farmer UI may show TrustScore breakdown (completion rate, complaint ratio, resolution, stability, response) but hide weight constants (SOE Enhancement 9)

Fraud Detection (MSDD Enhancement 8 + SOE Enhancement 8):

```
def detect_fraud(provider_id):
    # check: review pattern anomaly (same reviewer)
    # check: sudden rating spike (std_dev in 7 days > threshold)
    # check: IP correlation between reviewer and provider
    # if flagged → reduce trust weight, flag for admin
```

Commit 1: `feat: implement provider exposure fairness algorithm` **Commit 2:** `feat: add marketplace fraud detection engine`

Day 19 — Wednesday, March 19

Arpit

Task: Drift Enforcement + Risk Index Computation

```
Files to create:
  backend/app/services/ctis/drift_enforcer.py   ← clamp stage_offset to
max_allowed_drift
  backend/app/services/ctis/risk_calculator.py  ← risk_index = weather*0.7
+ farmer*0.3
```

Commit: `feat: implement drift enforcement and risk index computation`

Shivam

Task: ML model registry + versioning

```
Files to create:
  backend/app/api/v1/ml.py                 ← GET /ml/models, POST
/ml/models (admin)
  backend/app/services/ml/model_registry.py ← register, activate,
deactivate model versions
```

Commit: `feat: add ML model registry and versioning endpoints`

Day 20 — Thursday, March 20

Arpit

Task: Dead Letter Queue + Event failure handling

```
Files to modify:
  backend/app/services/event_dispatcher/db_dispatcher.py  ← add retry
logic, dead letter transition
  backend/app/api/v1/admin.py  ← add GET /admin/dead-letters, POST
/admin/dead-letters/{id}/retry
```

Commit: `feat: add dead letter queue handling and admin retry endpoint`

Prince

Task: Frontend — Action Logging UI + What-If Simulation page

```
Files to create:
  frontend/src/app/crops/[id]/log-action/page.tsx   ← log action form
  frontend/src/app/crops/[id]/simulate/page.tsx     ← what-if simulation
UI
  frontend/src/components/ActionForm.tsx             ← form with action
type, date, metadata
  frontend/src/components/SimulationResult.tsx       ← display projected
state after hypothetical action
```

Commit 1: `feat: add action logging form page` **Commit 2:** `feat: add what-if simulation page`

Day 21 — Friday, March 21

Ayush

Task: Admin governance APIs (rule management, kill switches)

```
Files to create:
  backend/app/api/v1/rules.py     ← CRUD for crop rule templates (admin
only) — rule_version_applied tracking (MSDD 1.4)
  backend/app/api/v1/features.py  ← GET/PUT feature flags (ml_enabled,
clustering_enabled, risk_prediction_enabled,
behavioral_adaptation_enabled) — ML Kill Switch (ML Enhancement 10)
```

ML Kill Switch (ML Enhancement 10): Admin may disable risk prediction influence, behavioral adaptation, and regional clustering. CTIS must fall back to deterministic rule engine when disabled. **Commit:** `feat: add admin rule management and feature flag endpoints`

Prince

Task: Frontend admin panel

```
Files to create:
  frontend/src/app/admin/page.tsx              ← admin dashboard
  frontend/src/app/admin/users/page.tsx        ← user management
  frontend/src/app/admin/providers/page.tsx    ← provider management
  frontend/src/components/DataTable.tsx        ← reusable table component
```

Commit: `feat: add admin panel pages with data tables`

Day 22 — Saturday, March 22

Arpit

Task: Notification / Alert + Recommendation Engine

```
Files to create:
  backend/app/models/alert.py                          ← alerts table
(MSDD Enhancement Sec 14: alert_id, crop_instance_id, alert_type,
severity, message, is_acknowledged, acknowledged_at, created_at)
  backend/app/models/recommendation.py                 ← recommendations
table (Patch Module 2, Sec 15: recommendation_id, crop_instance_id, type,
priority_rank, message_key, message_parameters JSONB, basis, valid_from,
valid_until, status, generated_at)
  backend/app/schemas/alert.py                         ← AlertResponse,
AlertAcknowledge
  backend/app/schemas/recommendation.py                ←
RecommendationResponse
  backend/app/services/notifications/__init__.py
  backend/app/services/notifications/alert_service.py  ← generate alerts
from events, throttling (max 3 per crop per 24h)
  backend/app/services/recommendations/__init__.py
  backend/app/services/recommendations/recommendation_engine.py ← compute
daily prioritized recommendations
  backend/app/api/v1/alerts.py                         ← GET /alerts, PUT
/alerts/{id}/acknowledge
  backend/app/api/v1/recommendations.py                ← GET
/crops/{id}/recommendations
  backend/alembic/versions/008_create_alerts_and_recommendations_tables.py
```

Alert Types: weather_alert, stress_alert, pest_alert, action_reminder, market_alert, harvest_approaching **Alert Throttling:** Max 3 alerts per crop per 24h. Configurable per alert type. **Recommendation Priority Formula (Patch Sec 15):**

```
def compute_priority(recommendation):
    urgency = URGENCY_WEIGHTS[recommendation.type]  # harvest_prep >
irrigation > fertilizer > general
    score = urgency * stage_criticality + risk_index * 0.4 +
```

```
days_until_deadline * -0.1
    return score  # top 3 surfaced per crop per day
```

Commit 1: `feat: implement notification and alert system` **Commit 2:** `feat: add recommendation engine with priority ranking`

Ayush

Task: Unit tests for auth module

```
Files to create:
  backend/tests/__init__.py
  backend/tests/conftest.py              ← test DB, test client fixtures
  backend/tests/test_auth.py             ← test register, login, JWT
validation
```

Commit: `test: add unit tests for authentication module`

WEEK 4 — Integration, Testing & Deployment (Mar 23–30)

Day 23 — Sunday, March 23

Arpit

Task: Offline sync endpoint

```
Files to create:
  backend/app/api/v1/sync.py                  ← POST /offline-sync (bulk
action array)
  backend/app/services/ctis/sync_service.py   ← validate density, order,
debounce replay
```

Temporal Anomaly Detection (MSDD 1.7.1): sync_service must detect:

- Excessive back-dated actions (action_effective_date far in the past)
- Excessive future-dated actions (beyond server_time + tolerance_limit)
- Large batch submission anomalies (>N actions in single sync)
- Monotonic counter resets (local_seq_no not strictly increasing per session)
- If anomaly_score > threshold: flag in abuse_flags table, notify admin **Commit:** `feat: implement offline sync endpoint with replay debouncing`

Ravi

Task: Service review + complaint endpoints + tests

```
Files to create:
  backend/app/api/v1/reviews.py     ← POST /reviews (after completed
request), complaint_category
  backend/tests/test_soe.py         ← test trust score, provider CRUD
```

Commit 1: `feat: implement service review and complaint endpoints` **Commit 2:** `test: add tests for SOE module`

Day 24 — Monday, March 24

Arpit

Task: Background event processor loop

```
Files to modify:
  backend/app/main.py  ← add startup event that launches asyncio
background task for process_pending_events()
  backend/app/services/event_dispatcher/db_dispatcher.py ← add polling
loop with asyncio.sleep
```

Commit: `feat: add background event processing loop on app startup`

Prince

Task: Frontend — Service Marketplace page

```
Files to create:
  frontend/src/app/services/page.tsx           ← browse providers
  frontend/src/app/services/request/page.tsx   ← create service request
  frontend/src/components/ProviderCard.tsx
  frontend/src/components/AlertBanner.tsx        ← notification/alert
display component
  frontend/src/app/alerts/page.tsx               ← alerts list page
```

Commit 1: `feat: add service marketplace and request pages` **Commit 2:** `feat: add alerts notification page and banner component`

Day 25 — Tuesday, March 25

Arpit

Task: Dockerfiles finalized + docker-compose with all services

```
Files to modify:
  docker-compose.yml   ← add volumes, env vars, healthchecks, network
  backend/Dockerfile   ← multi-stage build, production-ready
```

Commit: `deploy: finalize docker configuration for all services`

Shivam

Task: Media analysis stub + tests

```
Files to create:
  backend/app/services/media/analysis_service.py  ← stub CNN inference,
emit MediaAnalyzed event
  backend/tests/test_ml.py                        ← test risk predictor,
model registry
```

Commit 1: `feat: add media analysis service with event emission` **Commit 2:** `test: add tests for ML module`

Day 26 — Wednesday, March 26

Arpit

Task: Google Cloud Run deployment config

```
Files to create:
  backend/cloudbuild.yaml          ← Cloud Build config
  backend/.gcloudignore
  deploy/                          ← deployment scripts
  deploy/setup-cloud-sql.sh        ← Cloud SQL instance creation
  deploy/deploy-backend.sh         ← gcloud run deploy commands
```

Commit: `deploy: add cloud run deployment configuration`

Prince

Task: Frontend polish — responsive design, loading states, error handling + accessibility basics

```
Files to modify:
  frontend/src/components/*.tsx    ← add loading spinners, error
boundaries
  frontend/src/app/globals.css     ← responsive breakpoints, animations
Files to create:
  frontend/src/components/AccessibilityToggle.tsx ← toggle for large text
mode + high contrast mode (MSDD 7.1-7.2)
```

```
  frontend/src/hooks/useAccessibility.ts          ← read/save
accessibility_settings from user profile
```

Accessibility (MSDD Section 7):

- Large Text Mode: increase base font-size by 1.5× (MSDD 7.1)
- High Contrast Mode: enhanced contrast ratios for all UI elements (MSDD 7.2)
- Settings persisted in user.accessibility_settings JSONB
- *(Deferred: voice assist, WhatsApp mode, regional language support)* **Commit 1:** `feat: add responsive design and loading states to frontend` **Commit 2:** `feat: add basic accessibility toggle (large text, high contrast)`

Day 27 — Thursday, March 27

Arpit

Task: Cloud SQL setup + run Alembic migrations in cloud

```
Files to create/modify:
  deploy/run-migrations.sh         ← connect to Cloud SQL proxy, run
alembic upgrade
  backend/app/config.py            ← support Cloud SQL connection string
```

Commit: `deploy: add cloud sql migration scripts and config`

Ayush

Task: Integration tests — crop lifecycle flow

```
Files to create:
  backend/tests/test_crop_lifecycle.py  ← test: create crop → log action →
verify replay → submit yield
```

Commit: `test: add integration tests for crop lifecycle flow`

Day 28 — Friday, March 28

Arpit

Task: Cloud Storage integration for media

```
Files to modify:
  backend/app/services/media/upload_service.py  ← use google-cloud-storage
```

```
SDK, signed URLs
  backend/requirements.txt                        ← add google-cloud-storage
```

Commit: `feat: integrate google cloud storage for media uploads`

Shivam

Task: Seed data script + data initialization

```
Files to create:
  backend/scripts/seed_data.py     ← create sample crops, providers, rules
for demo
  backend/app/models/crop_rule_template.py ← add wheat, rice, cotton
templates
```

Commit: `chore: add seed data script with sample crop templates`

Ravi

Task: End-to-end SOE flow test

```
Files to create:
  backend/tests/test_soe_flow.py   ← create provider → create request →
accept → complete → review → trust recalc
```

Commit: `test: add end-to-end SOE workflow test`

Day 29 — Saturday, March 29

Arpit

Task: Final deployment to Cloud Run + bug fixes

```
Actions: deploy backend to Cloud Run, verify health endpoint, run
migrations
```

Commit 1: `fix: resolve deployment configuration issues` **Commit 2:** `deploy: successful cloud run deployment`

Prince

Task: Frontend — yield submission page + dashboard stats

```
Files to create:
  frontend/src/app/crops/[id]/yield/page.tsx
  frontend/src/components/YieldForm.tsx
  frontend/src/components/DashboardStats.tsx  ← total crops, active, at-
risk counts
```

Commit: `feat: add yield submission page and dashboard statistics`

Ayush

Task: Update README with complete setup instructions

```
Files to modify:
  README.md  ← detailed setup (prerequisites, env vars, how to run
locally, how to deploy)
```

Commit: `docs: update README with complete setup and deployment guide`

Day 30 — Sunday, March 30

Arpit

Task: Final integration checks, API documentation, frontend deployment

```
Files to create:
  docs/API.md           ← API endpoint documentation (auto-generated from
FastAPI /docs)
  frontend/next.config.js ← production config
```

Commit 1: `docs: add API documentation` **Commit 2:** `deploy: configure frontend for production build`

Shivam

Task: Add `.env.example` files, update requirements

```
Files to modify:
  backend/.env.example   ← all required env vars documented
  backend/requirements.txt ← final version pins
```

Commit: `docs: finalize environment configuration documentation`

Ravi

Task: Add project architecture docs

```
Files to create:
  docs/ARCHITECTURE.md   ← system architecture overview with component
descriptions
```

Commit: `docs: add system architecture documentation`

Prince

Task: Frontend README + component documentation

```
Files to create:
  frontend/README.md     ← how to run frontend, component list
```

Commit: `docs: add frontend documentation`

Commit Distribution Summary (Final)

Member	Total Commits	Days Active	Focus Areas
Arpit	~32	22/30	DB schema, Sowing Calendar, Replay Engine, Event Dispatcher, State Machine, Stress Engine, Deviation Tracker, Behavioral Adaptation, Recommendation Engine, Notification System, Deployment, Cloud, some Frontend
Ayush	~12	10/30	Auth, JWT, Idempotency/error middleware, Admin APIs (user + provider mgmt), ML Kill Switch, Router registration, Tests, README
Ravi	~13	10/30	SOE CRUD, Equipment CRUD, Labor CRUD, Trust Engine (decay, consistency), Exposure Fairness, Fraud Detector, Escalation Policy, Reviews, Tests
Prince	~12	10/30	All frontend pages (dashboard, crops, timeline, what-if, marketplace, admin, alerts), auth context, responsive polish, accessibility basics
Shivam	~12	10/30	ML module (confidence propagation), Weather API, Media handling (retention policy), Analytics tables (stress_history, regional_clusters, pest_alert_history), Crop rule seeds, Seed data, Tests

Total: ~81 commits across 30 days (avg 2.7/day — natural and incremental).

> ✅ Each non-lead member has at least **1 commit every 2 days** across their active windows. ✅ Arpit has ~40% of total commits. ✅ Other members each have ~15% of total commits.

Complete Feature Coverage Checklist

CTIS (Crop Timeline Intelligence System)

- ☑ CropInstance CRUD with pagination/filtering (Day 9 — Arpit)
- ☑ Seasonal Window Assignment logic (Day 9 — Arpit)
- ☑ Sowing Date Modification with full replay (Day 9 — Arpit)
- ☑ Action Logging with chronological validation (Day 10 — Arpit)
- ☑ Replay Engine with snapshot support + row locking + RecoveryRequired failsafe (Day 12 — Arpit)
- ☑ State Machine enforcement with RecoveryRequired state + event emission on transitions (Day 13 — Arpit)
- ☑ Stress Score Integration (Day 14 — Arpit)
- ☑ Deviation Profile Tracking (Day 14 — Arpit)
- ☑ Snapshot Manager (Day 15 — Arpit)
- ☑ What-If Simulation + API endpoint + deep copy enforcement (Day 16 — Arpit)
- ☑ Yield Submission + YieldVerificationScore + bio cap + Farmer/ML/Regional Truth separation (Day 17 — Arpit)
- ☑ Behavioral Adaptation with safeguards (max ±X days, reversible, expires at season end) (Day 17 — Arpit)
- ☑ Drift Enforcement (Day 19 — Arpit)
- ☑ Risk Index Computation (Day 19 — Arpit)
- ☑ Crop Rule Templates with version_id, effective_from_date (Day 15 — Arpit, seed data Day 8 — Shivam)
- ☑ Offline Sync with debouncing + temporal anomaly detection (Day 23 — Arpit)
- ☑ Recommendation Engine with priority ranking (Day 22 — Arpit)

Event Dispatcher

- ☑ Dispatcher Interface + DB implementation (Day 11 — Arpit)
- ☑ Event hash idempotency (Day 11 — Arpit)
- ☑ Event types catalog constants (CTIS/SOE/ML/Admin/Notification events) (Day 11 — Arpit)
- ☑ Replay trigger coordination (Day 13 — Arpit)
- ☑ Dead Letter Queue (Day 20 — Arpit)
- ☑ Background processing loop (Day 24 — Arpit)
- ☑ Crash recovery (auto-reset Processing → Created) (Day 24 — Arpit)

SOE (Service Orchestration Ecosystem)

- ☑ Provider CRUD with crop_specializations (Day 10 — Ravi)
- ☑ Equipment CRUD (Day 10 — Ravi)
- ☑ Labor model + CRUD (Day 4 — Ravi, MSDD 2.6)
- ☑ Service Request lifecycle + ServiceRequestEvent emission on state changes (Day 17 — Ravi, SOE Enhancement 7)
- ☑ Trust Score Engine with ConsistencyScore + temporal decay (Day 12 — Ravi, SOE Enhancement 2, 5)
- ☑ Exposure Fairness with regional saturation control + trust transparency (Day 18 — Ravi, SOE Enhancement 1, 9, 11)

- ☑ Fraud Detector with anti-fake review protection (Day 18 — Ravi, SOE Enhancement 8)
- ☑ Escalation Policy (Day 13 — Ravi)
- ☑ Reviews + Complaints with eligibility checks (Day 23 — Ravi)
- ☑ SOE isolation from CTIS ✓ (by design, SOE Enhancement 6)

ML & Intelligence

- ☑ Risk Predictor stub with confidence propagation (Day 13 — Shivam, ML Enhancement 2)
- ☑ Model Registry + Versioning (Day 19 — Shivam)
- ☑ Media Upload with retention policy + analysis status lifecycle (Day 14 — Shivam)
- ☑ Media Analysis stub (Day 25 — Shivam)
- ☑ Weather API integration (Day 14 — Shivam)
- ☑ ML Kill Switch via feature flags (Day 21 — Ayush, ML Enhancement 10)
- ☑ Feature Lineage Tracking in ml_training_audit (Day 5 — Shivam, ML Enhancement 8)

Auth & Admin

- ☑ JWT auth utilities (Day 2 — Ayush)
- ☑ Register/Login endpoints (Day 8 — Ayush)
- ☑ Role-based access control (Day 9 — Ayush)
- ☑ Error handling + idempotency middleware (Day 2 — Ayush)
- ☑ Router registration (Day 10 — Ayush)
- ☑ Admin user management (Day 15 — Ayush)
- ☑ Admin provider verify/suspend + audit log endpoints (Day 15 — Ayush)
- ☑ Rule template CRUD (Day 21 — Ayush)
- ☑ Feature flags (Day 21 — Ayush)

Frontend (Next.js)

- ☑ Project scaffold (Day 1 — Prince)
- ☑ Auth context + protected routes (Day 5 — Prince)
- ☑ Layout + navigation (Day 7 — Prince)
- ☑ Login/Register pages (Day 8 — Prince)
- ☑ Dashboard with crop cards (Day 11 — Prince)
- ☑ Crop list + detail + timeline (Day 16 — Prince)
- ☑ Action logging form (Day 20 — Prince)
- ☑ What-If simulation page (Day 20 — Prince)
- ☑ Admin panel (Day 21 — Prince)
- ☑ Service marketplace with trust score breakdown display (Day 24 — Prince)
- ☑ Alerts/notifications page (Day 24 — Prince)
- ☑ Yield submission page (Day 29 — Prince)
- ☑ Responsive polish (Day 26 — Prince)
- ☑ Accessibility basics: large text mode + high contrast toggle (Day 26 — Prince, MSDD 7.1-7.2)

Database

- ☑ Users table with accessibility_settings + soft delete fields (Day 3 — Arpit)

- ☑ CTIS tables: crop_instances, action_logs, snapshots, deviation_profiles, yield_records (Day 4 — Arpit)
- ☑ SOE tables: service_providers, equipment, labor, service_requests, service_reviews, provider_availability, service_request_events (Day 4 — Ravi)
- ☑ Event tables: event_log (Day 5 — Arpit)
- ☑ Admin tables: admin_audit_log, feature_flags, abuse_flags (Day 5 — Arpit)
- ☑ ML/Analytics tables: ml_models, ml_training_audit, stress_history, regional_clusters, pest_alert_history (Day 5 — Shivam)
- ☑ Media tables: media_files with scheduled_deletion_at + analysis_status (Day 5 — Shivam)
- ☑ Regional sowing calendars (Day 5 — Arpit)
- ☑ Alerts + Recommendations tables (Day 22 — Arpit)
- ☑ Crop rule templates (Day 15 — Arpit)

Notification & Recommendation System

- ☑ Alert model + table with alert_type enum (weather, stress, pest, action_reminder, market, harvest) (Day 22 — Arpit)
- ☑ Alert generation from events (Day 22 — Arpit)
- ☑ Alert throttling (max 3 per crop per 24h, configurable per type) (Day 22 — Arpit)
- ☑ Alert API endpoints with acknowledgment tracking (Day 22 — Arpit)
- ☑ Alert + Recommendation Pydantic schemas (Day 22 — Arpit)
- ☑ Recommendation model + table (Day 22 — Arpit)
- ☑ Recommendation engine with priority formula (Day 22 — Arpit)
- ☑ Recommendation API endpoint (Day 22 — Arpit)
- ☑ Frontend alert UI (Day 24 — Prince)

Deployment & Infrastructure

- ☑ Docker + docker-compose (Day 7 — Arpit)
- ☑ .dockerignore for backend (Day 7 — Arpit)
- ☑ Dockerfiles finalized (Day 25 — Arpit)
- ☑ Cloud Run config (Day 26 — Arpit)
- ☑ Cloud SQL setup (Day 27 — Arpit)
- ☑ Cloud Storage integration (Day 28 — Arpit)
- ☑ Final deployment (Day 29 — Arpit)

Testing

- ☑ Auth tests (Day 22 — Ayush)
- ☑ SOE tests (Day 23 — Ravi)
- ☑ ML tests (Day 25 — Shivam)
- ☑ Crop lifecycle integration test (Day 27 — Ayush)
- ☑ SOE end-to-end test (Day 28 — Ravi)

Documentation

- ☑ README.md (Day 1 — Arpit, updated Day 29 — Ayush)
- ☑ API docs (Day 30 — Arpit)

- ☑ Architecture docs (Day 30 — Ravi)
- ☑ Frontend docs (Day 30 — Prince)
- ☑ Env config docs (Day 30 — Shivam)

Critical Files Reference (from TDD/MSDD)

Use these as implementation reference:

- **DB Schema:** TDD Section 2 (all `CREATE TABLE` statements)
- **Replay Algorithm:** TDD Section 4.4 (pseudocode)
- **Replay Failure Handling:** MSDD Section 1.18 (RecoveryRequired state)
- **Event Dispatcher:** TDD Section 3.3–3.12
- **Event Types Catalog:** MSDD Section 3.3.1 (formal taxonomy)
- **Trust Score Formula:** TDD Section 5.5 + SOE Enhancement 2, 5
- **Stress Integration:** TDD Section 4.7
- **Drift Enforcement:** TDD Section 4.6
- **State Machine:** MSDD Section 1.5
- **Deviation Profile:** MSDD Section 1.9.1
- **Behavioral Adaptation:** MSDD Section 4.2 Layer 3 + ML Enhancement 6
- **Confidence Propagation:** ML Enhancement 2
- **ML Kill Switch:** ML Enhancement 10
- **Feature Lineage Tracking:** ML Enhancement 8
- **Model Drift Detection:** ML Enhancement 5
- **Escalation Policy:** MSDD Enhancement 3
- **Fraud Detection:** MSDD Enhancement 8
- **Marketplace Fairness:** SOE Enhancement 1
- **Trust Decay:** SOE Enhancement 2
- **ConsistencyScore:** SOE Enhancement 5
- **SOE–CTIS Isolation:** SOE Enhancement 6
- **Service Request Audit Trail:** SOE Enhancement 7
- **Trust Transparency:** SOE Enhancement 9
- **Regional Saturation Control:** SOE Enhancement 11
- **Marketplace Risk Flags:** SOE Enhancement 12
- **Notification Architecture:** MSDD Enhancement Section 14
- **Recommendation Engine:** MSDD Patch Module 2, Section 15
- **Seasonal Window Assignment:** MSDD 1.9 + Patch Module 5
- **Temporal Anomaly Detection:** MSDD Section 1.7.1
- **Offline Abuse Detection:** MSDD Patch Section 4.1
- **Weather Integration:** MSDD Section 4.8
- **Labor Model:** MSDD Section 2.6
- **Equipment Table:** TDD Section 5.5.2
- **Pest Alert History:** MSDD Section 6 Enhancement
- **Stress History:** TDD Section 5.6.1
- **Regional Clusters:** TDD Section 5.6.2
- **Yield Verification:** MSDD Section 4.3, 1.12.1

- **Accessibility:** MSDD Section 7
- **Soft Delete Policy:** MSDD Section 5.10
- **Idempotency Handling:** MSDD Section 8.13
- **API Contract (all endpoints):** MSDD Section 8

Explicitly Deferred (Post-MVP per MSDD 13.14)

The following are documented but explicitly **not included** in this 30-day plan per MVP scope:

Feature	Source	Reason
Edge AI (TFLite on-device)	MSDD 4.5	Phase 8 — Optional Advanced
Regional Clustering batch job	MSDD 4.2 Layer 4	Phase 7 — Advanced
WhatsApp bot webhook service	MSDD 7.12	Phase 5 — separate service
Voice note intent parsing	MSDD 6.8	Complex NLP — post-MVP
Voice Assist Mode	MSDD 7.11	Requires speech-to-text — post-MVP
Video processing worker (separate Cloud Run)	MSDD 4.6	Phase 8 — async pipeline
Video CNN/YOLO-lite processing	MSDD 4.6	Requires trained models — deferred
Market data integration (ARIMA)	MSDD 4.7	Phase 6+ — no API source yet
Long-Horizon Forecast (ML Enhancement 4)	ML Enhancement 4	No market data endpoint
Model Drift Monitoring	ML Enhancement 5	Requires deployed models
Edge AI Confidence Degradation	ML Enhancement 9	Requires Edge AI
Cluster Stability Validation	ML Enhancement 7	Requires cluster data
Storage & Transport Suggestion Module	MSDD 2.3	Not critical for prototype
Regional Language Support (i18n)	MSDD 7.6	Complex i18n — post-MVP
Push Notifications (FCM/APNs)	MSDD Enhancement Sec 14	Requires mobile app config
Multi-region deployment	Deployment	Post-prototype scale
PgBouncer / connection pooling	Infrastructure	Only needed at >200 users